

Lab 1: Rudy - A Small Web Server

Sevastian Bajenov

August 30, 2026

1 Introduction

In this laboratory work, my main goal was to implement a basic 2-tier (client-server) architecture. This type of architecture is very common and should be studied as the basis of more advanced distributed systems. Furthermore, I had to optimize the server by introducing the concept of concurrency in terms of handling incoming client requests.

2 Main problems and solutions

First of all, I completed the implementation of the simplified http parser. Its main goals are to parse the incoming GET request, extract the URI, version, headers, and body in a pretty straightforward way. The parser file also includes a method for returning generic OK response and sending a GET request.

One of the optimizations that I applied to the proposed source code was using tail recursion. For example, the following method for request URI extraction:

```
request_uri([32|R0])->
  {[], R0};
request_uri([C|R0]) ->
  {Rest, R1} = request_uri(R0),
  {[C|Rest], R1}.
```

was transformed into the following better version:

```
request_uri(List) ->
  request_uri(List, []).

request_uri([32 | R0], Acc) ->
  {reverse(Acc), R0};
request_uri([C | R0], Acc) ->
  request_uri(R0, [C | Acc]).
```

This version enables the compiler to optimize the code processing so that the recursive calls do not get stacked during method execution.

After that, I proceeded with different server implementations. I started with the most simple sequential server model that served one request at a time. The handler method which accepts socket as a parameter and listens for incoming connections looks like this:

```
handler(Listen) ->
  case gen_tcp:accept(Listen) of
    {ok, Client} ->
      request(Client),
      handler(Listen),
      ok;
    {error, _} ->
      error
  end.
```

This server is not that efficient; therefore, the first thing that I tried to implement is spawning a separate process for each incoming request:

```
handler(Listen) ->
  case gen_tcp:accept(Listen) of
    {ok, Client} ->
      spawn(fun() -> request(Client) end),
      handler(Listen),
      ok;
    {error, _} ->
      error
  end.
```

The second approach was to have a pool of handlers that are being created in the server init method:

```
manager_init(Port, NumAcceptors) ->
  process_flag(trap_exit, false),
  Opt = [list, {active, false}, {reuseaddr, true}],

  case gen_tcp:listen(Port, Opt) of
    {ok, Listen} ->
      [
        spawn_link(fun() -> acceptor(Listen) end)
        || _ <- lists:seq(1, NumAcceptors)
      ],
      receive
        after infinity -> ok
      end
```

```

        end;
    {error, Reason} ->
        {error, Reason}
    end.

```

Finally, I tried to use these server variants in both single and multi core Erlang modes to see the results. Their detailed explanations can be found in the following section.

3 Evaluation

All of the evaluation tests were run via the Erlang shell using either "erl +S 1:1" or "erl +S 14:14" (I have 14 available schedulers verified via the system info command) for single and multi-core modes respectively. The main metric was Requests Per Second (RPS).

The first test focuses on verifying the impact of the http parser on the overall server performance by introducing an artificial delay (see Table 1 and 2). The results confirm two things: the parser overhead is negligible and is incomparable with the delay; the server's performance decreases once the backlog limit is hit (default 5 pending connections in the queue).

Clients	Aggregate RPS
1	23.87
2	24.38
4	24.38
8	20.05

Table 1: With timer:sleep(40) in reply/1

Clients	Aggregate RPS
1	5305.88
2	7072.39
4	12458.34
8	746.50

Table 2: Without artificial delay in reply/1

The next three tests evaluate the performance of the three versions of the server developed in the context of this lab.

Clients	Aggregate RPS (single-core)	Aggregate RPS (multicore)
1	23.76	23.74
2	24.37	24.37
4	24.38	24.37
8	19.90	20.00

Table 3: Aggregate requests per second for the sequential server

Clients	Aggregate RPS (single-core)	Aggregate RPS (multicore)
1	23.87	23.82
2	47.55	47.40
4	94.92	94.72
8	129.16	190.91
16	258.83	310.60

Table 4: Aggregate requests per second for the spawn-per-connection server

Clients	Aggregate RPS (single-core)	Aggregate RPS (multicore)
1	23.87	23.84
2	47.51	47.55
4	94.88	95.08
8	97.51	97.44
16	86.16	85.70

Table 5: Aggregate requests per second for the server with a pool of four acceptors

It is clear from the results that the proposed optimizations significantly enhance the throughput of the server. However, the benefit of using multi core approach can be observed only in the spawn-per-connection approach. The pool of workers creates the artificial limit for the number of request acceptors, therefore it does not show better results with several cores available.

4 Conclusions

In this laboratory work I had the opportunity to implement client-server application in Erlang and familiarize myself with the language and its principles. I also explored both sequential and concurrent approaches in server design. For example, the spawn-per-connection solution demonstrated great results in the benchmark, however it is still not ideal. Although, Erlang processes are lightweight and easy to spawn, if the number of clients becomes significantly large, this approach might not be applicable.